



Guía n°

Configurar MSAL en el frontend

ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Guía n°

Nombre de la guía

Sigla	Asignatura	Experiencia de Aprendizaje
	Cloud Native I	
Tiempo	Modalidad de Trabajo	Indicadores de Logro

Contexto

MSAL (Microsoft Authentication Library) es la librería oficial de Microsoft para:

- **Iniciar sesión** con una cuenta de Microsoft Entra ID (antes Azure AD).
- **Obtener tokens** (ID Token y Access Token).
- **Usar esos tokens** para llamar APIs protegidas (Microsoft Graph o nuestras APIs).

MSAL hace todo el flujo OAuth2/OIDC por dentro; en el front solo configuramos:

1. **Quién soy yo** → clientId (ID de la app SPA).
2. **En qué tenant estoy** → authority con el tenantId.
3. **A dónde volver después del login** → redirectUri.
4. **A qué API quiero llamar y con qué permisos (scopes)** → protectedResourceScopes.

Requisitos previos

Antes de integrar **Microsoft Authentication Library (MSAL)** en nuestro frontend, es **obligatorio** que los alumnos hayan completado el tutorial **“Creando una aplicación para usuarios externos”**, ya que ahí se configura la base de identidad que MSAL utilizará.

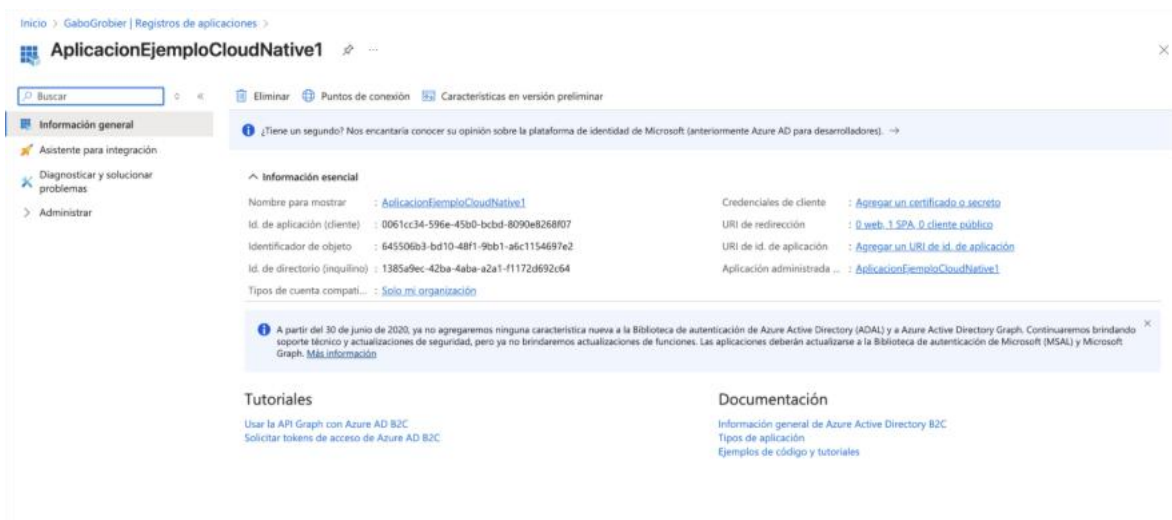
Este registro es indispensable porque MSAL **necesita conocer los datos de identidad de la aplicación** (clientId, tenant, redirectUri), los cuales solo existen después de registrar la app en Microsoft Entra ID.

¿Por qué se debe registrar la aplicación antes de integrar MSAL?

Porque MSAL solo puede funcionar si sabe:

- **Quién es tu aplicación** → *Application (client) ID*
- **En qué tenant vive** → *Directory (tenant) ID*
- **A dónde debe volver el usuario después del login** → *Redirect URI (SPA)*
- **Para quién muestra el login** → *Tipo de cuenta compatible (single/multi-tenant)*

Toda esta información se obtiene únicamente después de seguir el tutorial de registro.



Comenzamos

Instalar MSAL en el proyecto Angular

Primero lo que debemos hacer es instalar las librerías de MSAL

- `npm install @azure/msal-browser @azure/msal-angular`

¿Por qué se instalan dos librerías?

- **@azure/msal-browser** → maneja toda la autenticación en el navegador.
- **@azure/msal-angular** → integra MSAL con Angular (guards, interceptors, servicios).



Configurar los valores de Entra ID en environment.ts

Debemos colocar los valores de la app registrada en nuestro environment.ts

```
// src/environments/environment.ts
export const environment = {
  production: false,

  azure: {
    clientId: 'PON_AQUI_TU_CLIENT_ID',
    tenantId: 'PON_AQUI_TU_TENANT_ID',
    authority: 'https://login.microsoftonline.com/PON_TU_TENANT_ID',
    redirectUri: 'http://localhost:4200', // Debe coincidir con lo configurado en Entra

    // Sólo si usarás API con scopes:
    protectedResourceScopes: ['api://TU_API_ID_URI/TU_SCOPE']
  },

  // En caso de tener backend:
  apiUrl: 'http://localhost:8080'
};
```

Crear la instancia de MSAL (factory function)

MSAL necesita una instancia de PublicClientApplication.

Crear un archivo (o incluirlo en main.ts):



```
import { PublicClientApplication, IPublicClientApplication
, LogLevel } from '@azure/msal-browser';
import { environment } from './environments/environment';

export function msalInstanceFactory():
IPublicClientApplication
return new PublicClientApplication({
  auth:
    clientId: environment.azure.clientId,
    authority: environment.azure.authority,
    redirectUri: environment.azure.redirectUri
  },
  cache:
    cacheLocation: 'localStorage'
  }
});
```

¿Por qué esto es necesario?

Porque MSAL Angular exige que registremos esta instancia como proveedor global.

Configurar MSAL Interceptor para las peticiones HTTP

Si el proyecto consume un API protegido, necesitamos un interceptor.



```
import { MsalInterceptorConfiguration, InteractionType } from '@azure/msal-angular';

export function msalInterceptorConfigFactory(): MsalInterceptorConfiguration {
  const protectedResourceMap = new Map<string, Array<string>>([
    [`${environment.apiUrl}/*`, environment.azure.protectedResourceScopes]
  ]);

  return {
    interactionType: InteractionType.Redirect,
    protectedResourceMap
  };
}
```

¿Para qué sirve?

Cada vez que Angular haga peticiones al backend, MSAL agregará automáticamente:

Authorization: Bearer <token>

Registrar MSAL en el bootstrap de Angular (main.ts)

En un proyecto standalone Angular (Angular 16+), el registro se hace así:



```
import { bootstrapApplication } from '@angular/platform-browser';
import { AppComponent } from './app/app.component';

import { HTTP_INTERCEPTORS, provideHttpClient, withInterceptorsFromDi } from '@angular/common/http';
import {
  MSAL_GUARD_CONFIG,
  MSAL_INSTANCE,
  MSAL_INTERCEPTOR_CONFIG,
  MsalInterceptor,
  MsalGuard,
  MsalService,
  MsalBroadcastService
} from '@azure/msal-angular';

import { provideRouter, Routes } from '@angular/router';
import { msalInstanceFactory } from './msal-config';
import { msalInterceptorConfigFactory } from './msal-interceptor-config';

const routes: Routes = [
  { path: '', component: HomeComponent, canActivate: MsalGuard },
  { path: '**', redirectTo: '' },
];

bootstrapApplication(AppComponent, {
  providers: [
    provideRouter(routes),
    provideHttpClient(withInterceptorsFromDi()),

    { provide: MSAL_INSTANCE, useFactory: msalInstanceFactory },
    { provide: MSAL_INTERCEPTOR_CONFIG, useFactory: msalInterceptorConfigFactory },

    { provide: HTTP_INTERCEPTORS, useClass: MsalInterceptor, multi: true },

    MsalGuard,
    MsalService,
    MsalBroadcastService
  ]
});
```

¿Por qué hacemos esto?

- **MsalGuard** → protege rutas, obliga a login antes de entrar.
- **MsalInterceptor** → agrega tokens en las peticiones HTTP.
- **MsalService** → permite login/logout desde cualquier componente.

Agregar botones de Login / Logout en el componente principal



```
import { Component } from '@angular/core';
import { MsalService } from '@azure/msal-angular';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html'
})
export class AppComponent {
  constructor(private msa: MsalService) {}

  isLoggedIn(): boolean {
    return this.msal.instance.getAllAccounts().length > 0;
  }

  login(): void {
    this.msal.loginRedirect();
  }

  logout(): void {
    this.msal.logoutRedirect();
  }
}
```

App.component.html

```
<button *ngIf="!isLoggedIn()" (click)="login()">Iniciar sesión</button>
<button *ngIf="isLoggedIn()" (click)="logout()">Cerrar sesión</button>

<router-outlet></router-outlet>
```

Probar la autenticación

1. Ejecutar el proyecto:



ng serve

2. Abrir:
<http://localhost:4200>
3. Presionar **Iniciar sesión**.
4. Serás redirigido a la pantalla de Microsoft Entra ID.
5. Luego de iniciar sesión, el navegador vuelve al frontend con el usuario autenticado.
6. Ahora podrás acceder a:
 - a. rutas protegidas por MsalGuard
 - b. APIs protegidas con scopes

Linkografía

Conceptos básicos de registro de aplicaciones

<https://learn.microsoft.com/azure/active-directory/develop/quickstart-register-app>



Registro de una aplicación para SPA (Single-Page Application)

<https://learn.microsoft.com/azure/active-directory/develop/scenario-spa-app-registration>

. Configurar autenticación y URIs de redirección

<https://learn.microsoft.com/azure/active-directory/develop/howto-add-app-roles-portal>

Exponer un API y crear scopes (si usarán backend protegido)

<https://learn.microsoft.com/azure/active-directory/develop/quickstart-configure-app-expose-web-apis>

Documentación oficial de MSAL (Microsoft Authentication Library)

¿Qué es MSAL?

<https://learn.microsoft.com/azure/active-directory/develop/msal-overview>

Conceptos básicos de tokens, consentimientos y permisos

<https://learn.microsoft.com/azure/active-directory/develop/v2-permissions-and-consent>

MSAL Browser (base usada por Angular)

Introducción a @azure/msal-browser

<https://learn.microsoft.com/azure/active-directory/develop/msal-js-browser-overview>

Guía de configuración de aplicaciones en MSAL Browser

<https://learn.microsoft.com/azure/active-directory/develop/tutorial-v2-javascript-spa>



MSAL Angular – Integración paso a paso

Documentación principal de @azure/msal-angular

<https://learn.microsoft.com/azure/active-directory/develop/msal-angular>

Iniciar sesión en Angular (MsalService + loginRedirect/loginPopup)

<https://learn.microsoft.com/azure/active-directory/develop/msal-login>

Proteger rutas con MsalGuard

<https://learn.microsoft.com/azure/active-directory/develop/msal-angular#protect-routes-using-the-msalguard>

Usar MsalInterceptor para llamar APIs con tokens

<https://learn.microsoft.com/azure/active-directory/develop/msal-angular#protect-web-api-calls-using-the-msalinterceptor>

Inicializar MSAL Angular (MsalModule / bootstrap)

<https://learn.microsoft.com/azure/active-directory/develop/msal-angular#initializing-msal-angular>

Autenticación SPA – Flujo de autorización (OIDC/OAuth2)

Flujo completo de autorización con código para SPAs

<https://learn.microsoft.com/azure/active-directory/develop/v2-oauth2-auth-code-flow>

Microsoft Graph (si los alumnos usan APIs de Microsoft)

Permisos y tokens para Microsoft Graph

<https://learn.microsoft.com/graph/auth/auth-concepts>